

University Of Asia Pacific



Technical Report on

Tic Tac Toe with Minimax and Alpha-Beta Pruning

(Implementation of an AI-Powered Unbeatable Tic Tac Toe Game with Optimal Move Calculation)

Course Title: Artificial Intelligence & Expert System Lab

Course Code: CSE 404

Prepared for:
Bidita Sarkar Diba
Lecturer, Dept. of CSE
University of Asia
Pacific

Prepared by:
Abdullah Al-Mamun
ID No: 22101158
&
Anika Nower Nabila
ID No: 22101152

Date: 23 September 2025

Department of Computer Science and Engineering
University of Asia Pacific

Tic Tac Toe Game with Minimax and Alpha-Beta Pruning

i. Problem Title

Implementation of Tic Tac Toe with Artificial Intelligence using Minimax Algorithm and Alpha-Beta Pruning

ii. Problem Description

The aim of this project is to design and implement a **Tic Tac Toe game** where a human player can compete against an AI, or the AI can play against itself. This project focuses on integrating **Artificial Intelligence** to make optimal moves using **Minimax Algorithm** along with **Alpha-Beta Pruning** to optimize computational efficiency.

Tic Tac Toe is a simple two-player game played on a 3×3 grid. The first player uses 'X' and the second uses 'O'. Players take turns placing their symbols in empty cells. The player who aligns three of their symbols either horizontally, vertically, or diagonally wins the game.

Key Objectives of the Project:

1. Develop an interactive **graphical user interface (GUI)** for Tic Tac Toe using Python Tkinter.
2. Implement AI logic that can play **unbeatable moves** using Minimax.
3. Optimize AI performance by implementing **Alpha-Beta Pruning** to reduce unnecessary computations.
4. Provide multiple game modes:
 - **Human vs Computer**
 - **Computer vs Computer**
5. Enhance user experience with dynamic highlights, status updates, and interactive buttons.

Why this is important:

- Demonstrates application of AI in a simple board game.
- Shows the difference between standard Minimax and Alpha-Beta Pruning in practice.
- Combines programming, algorithms, and UI design in one project.

iii. Tools and Languages Used

Tool / Language	Purpose / Usage
Python 3.x	Core programming language for the game logic and AI implementation.
Tkinter	GUI library to create interactive buttons, labels, and frames.
Math	Provides mathematical constants and infinity for Minimax evaluation.
Random	To make the first AI move random for variety.
Algorithms	Minimax for AI decision-making and Alpha-Beta Pruning for optimization.

Additional Features Implemented:

- Dynamic button colors and hover effects for better UI.
- Status label to indicate the current player and game results.
- Highlighting the winning line in green for better visualization.

iii. Code

```
import tkinter as tk
from tkinter import messagebox
import math
import random

# --- Constants ---
EMPTY = ' '
PLAYER_X = 'X'
PLAYER_O = 'O'
BOARD_SIZE = 3
AI_DELAY = 500
BUTTON_FONT = ('Arial', 24, 'bold')
STATUS_FONT = ('Arial', 14, 'bold')
BG_COLOR = '#1e1e1e'
BUTTON_BG = '#2e2e2e'
BUTTON_FG_X = '#e74c3c'
BUTTON_FG_O = '#3498db'
HOVER_BG = '#3e3e3e'
WIN_LINE_BG = '#27ae60'

class TicTacToeApp:
    """Modern Tic Tac Toe game with Human vs AI and AI vs AI using
    Minimax with Alpha-Beta pruning."""
```

```

def __init__(self, root):
    self.root = root
    self.root.title("Tic Tac Toe")
    self.root.configure(bg=BG_COLOR)

    self.board = initial_state()
    self.current_player = PLAYER_X
    self.buttons = [[None]*BOARD_SIZE for _ in range(BOARD_SIZE)]
    self.game_mode = "human_vs_computer"
    self.first_move_done = False

    self.create_widgets()

# --- UI Setup ---
def create_widgets(self):
    # Board Frame
    board_frame = tk.Frame(self.root, bg=BG_COLOR)
    board_frame.pack(pady=20)

    for i in range(BOARD_SIZE):
        for j in range(BOARD_SIZE):
            btn = tk.Button(
                board_frame,
                text=EMPTY,
                font=BUTTON_FONT,
                width=5,
                height=2,
                bg=BUTTON_BG,
                fg='white',
                activebackground=HOVER_BG,
                command=lambda i=i, j=j: self.make_move(i, j)
            )
            btn.grid(row=i, column=j, padx=5, pady=5)
            self.buttons[i][j] = btn

    # Status Label
    self.status_label = tk.Label(self.root, text="Player X's
turn", font=STATUS_FONT, bg=BG_COLOR, fg='white')
    self.status_label.pack(pady=10)

```

```

    # Control Buttons
    control_frame = tk.Frame(self.root, bg=BG_COLOR)
    control_frame.pack(pady=10)
    tk.Button(control_frame, text="Human vs Computer",
command=self.set_human_vs_computer).grid(row=0, column=0, padx=5)
    tk.Button(control_frame, text="Computer vs Computer",
command=self.set_computer_vs_computer).grid(row=0, column=1, padx=5)
    tk.Button(control_frame, text="Reset",
command=self.reset_game).grid(row=0, column=2, padx=5)

    # --- Game Mode Handlers ---
    def set_human_vs_computer(self):
        self.game_mode = "human_vs_computer"
        self.reset_game()

    def set_computer_vs_computer(self):
        self.game_mode = "computer_vs_computer"
        self.reset_game()
        self.root.after(AI_DELAY, self.play_computer_vs_computer)

    def reset_game(self):
        self.board = initial_state()
        self.current_player = PLAYER_X
        self.first_move_done = False
        for i in range(BOARD_SIZE):
            for j in range(BOARD_SIZE):
                self.buttons[i][j].config(text=EMPTY, state=tk.NORMAL,
bg=BUTTON_BG, fg='white')
        self.status_label.config(text="Player X's turn")

    # --- Gameplay ---
    def make_move(self, i, j):
        if terminal(self.board) or self.board[i][j] != EMPTY:
            return

        # Update board
        self.board = result(self.board, (i, j))
        self.update_button(i, j)

```

```

    # Check winner
    winner_player = winner(self.board)
    if winner_player:
        self.status_label.config(text=f"{winner_player} wins!")
        self.highlight_winner(winner_player)
        self.disable_buttons()
    elif terminal(self.board):
        self.status_label.config(text="It's a draw!")
    else:
        self.current_player = player(self.board)
        self.status_label.config(text=f"Player
{self.current_player}'s turn")
        if self.game_mode == "human_vs_computer" and
self.current_player == PLAYER_0:
            self.root.after(AI_DELAY, self.computer_move)

    def update_button(self, i, j):
        symbol = self.current_player
        fg_color = BUTTON_FG_X if symbol == PLAYER_X else BUTTON_FG_O
        self.buttons[i][j].config(text=symbol, state=tk.DISABLED,
fg=fg_color)

    def computer_move(self):
        if terminal(self.board):
            return
        if not self.first_move_done:
            move = random.choice(list(actions(self.board)))
            self.first_move_done = True
        else:
            _, move = minimax(self.board, math.inf, float('-inf'),
float('inf'), True, PLAYER_0)

        if move:
            self.make_move(*move)

    def play_computer_vs_computer(self):
        if terminal(self.board):
            return

```

```

current_player = self.current_player
if not self.first_move_done:
    move = random.choice(list(actions(self.board)))
    self.first_move_done = True
else:
    _, move = minimax(self.board, math.inf, float('-inf'),
float('inf'), True, current_player)

    if move:
        self.make_move(*move)

    self.current_player = PLAYER_O if current_player == PLAYER_X
else PLAYER_X
    self.root.after(AI_DELAY, self.play_computer_vs_computer)

# --- Utility ---
def disable_buttons(self):
    for i in range(BOARD_SIZE):
        for j in range(BOARD_SIZE):
            self.buttons[i][j].config(state=tk.DISABLED)

def highlight_winner(self, player_symbol):
    """Highlight the winning line."""
    # Check rows
    for i in range(BOARD_SIZE):
        if all(self.board[i][j] == player_symbol for j in
range(BOARD_SIZE)):
            for j in range(BOARD_SIZE):
                self.buttons[i][j].config(bg=WIN_LINE_BG)
            return
    # Check columns
    for j in range(BOARD_SIZE):
        if all(self.board[i][j] == player_symbol for i in
range(BOARD_SIZE)):
            for i in range(BOARD_SIZE):
                self.buttons[i][j].config(bg=WIN_LINE_BG)
            return
    # Check diagonals

```

```

        if all(self.board[i][i] == player_symbol for i in
range(BOARD_SIZE)):
            for i in range(BOARD_SIZE):
                self.buttons[i][i].config(bg=WIN_LINE_BG)
            return
        if all(self.board[i][BOARD_SIZE-1-i] == player_symbol for i in
range(BOARD_SIZE)):
            for i in range(BOARD_SIZE):
                self.buttons[i][BOARD_SIZE-1-i].config(bg=WIN_LINE_BG)
            return

# --- Game Logic ---
def initial_state():
    return [[EMPTY for _ in range(BOARD_SIZE)] for _ in
range(BOARD_SIZE)]

def player(board):
    count_x = sum(row.count(PLAYER_X) for row in board)
    count_o = sum(row.count(PLAYER_O) for row in board)
    return PLAYER_O if count_x > count_o else PLAYER_X

def actions(board):
    return {(i, j) for i in range(BOARD_SIZE) for j in
range(BOARD_SIZE) if board[i][j] == EMPTY}

def result(board, action):
    i, j = action
    if board[i][j] != EMPTY:
        raise Exception("Invalid move")
    new_board = [row[:] for row in board]
    new_board[i][j] = player(board)
    return new_board

def winner(board):
    for i in range(BOARD_SIZE):

```



```

        if all(board[i][j] == PLAYER_X for j in range(BOARD_SIZE)):
            return PLAYER_X
        if all(board[i][j] == PLAYER_O for j in range(BOARD_SIZE)):
            return PLAYER_O
        if all(board[j][i] == PLAYER_X for j in range(BOARD_SIZE)):
            return PLAYER_X
        if all(board[j][i] == PLAYER_O for j in range(BOARD_SIZE)):
            return PLAYER_O
        if all(board[i][i] == PLAYER_X for i in range(BOARD_SIZE)) or
all(board[i][BOARD_SIZE-1-i] == PLAYER_X for i in range(BOARD_SIZE)):
            return PLAYER_X
        if all(board[i][i] == PLAYER_O for i in range(BOARD_SIZE)) or
all(board[i][BOARD_SIZE-1-i] == PLAYER_O for i in range(BOARD_SIZE)):
            return PLAYER_O
        return None

def terminal(board):
    return winner(board) is not None or all(cell != EMPTY for row in
board for cell in row)

def minimax(board, depth, alpha, beta, maximizing_player,
player_symbol):
    if terminal(board) or depth == 0:
        return evaluate_board(board, player_symbol), None

    best_move = None
    if maximizing_player:
        max_eval = float("-inf")
        for action in actions(board):
            eval_score, _ = minimax(result(board, action), depth-1,
alpha, beta, False, player_symbol)
            if eval_score > max_eval:
                max_eval = eval_score
                best_move = action
            alpha = max(alpha, max_eval)
        if beta <= alpha:
            break

```

```

        return max_eval, best_move
    else:
        min_eval = float("inf")
        for action in actions(board):
            eval_score, _ = minimax(result(board, action), depth-1,
alpha, beta, True, player_symbol)
            if eval_score < min_eval:
                min_eval = eval_score
                best_move = action
            beta = min(beta, min_eval)
            if beta <= alpha:
                break
        return min_eval, best_move

def evaluate_board(board, player_symbol):
    opponent = PLAYER_X if player_symbol == PLAYER_O else PLAYER_O
    if winner(board) == player_symbol:
        return 1
    elif winner(board) == opponent:
        return -1
    return 0

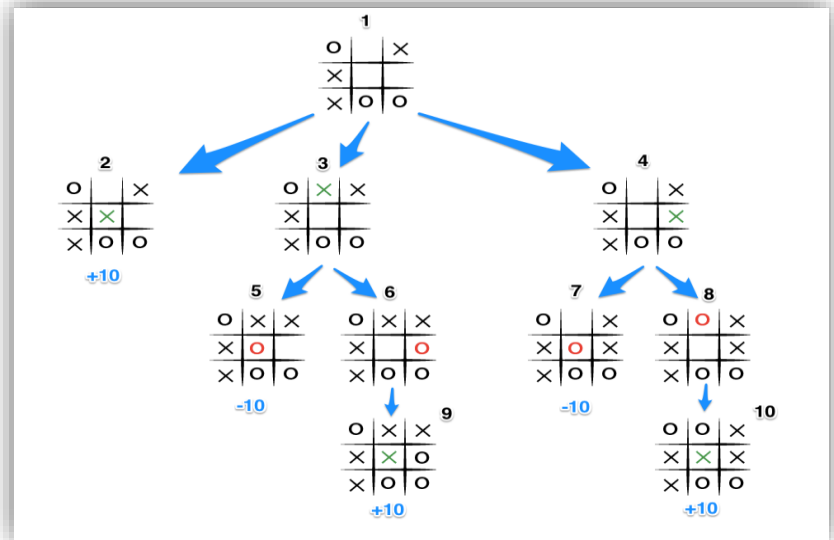
# --- Run Game ---
if __name__ == "__main__":
    root = tk.Tk()
    app = TicTacToeApp(root)
    root.mainloop()

```

iv. Game Tree Diagram

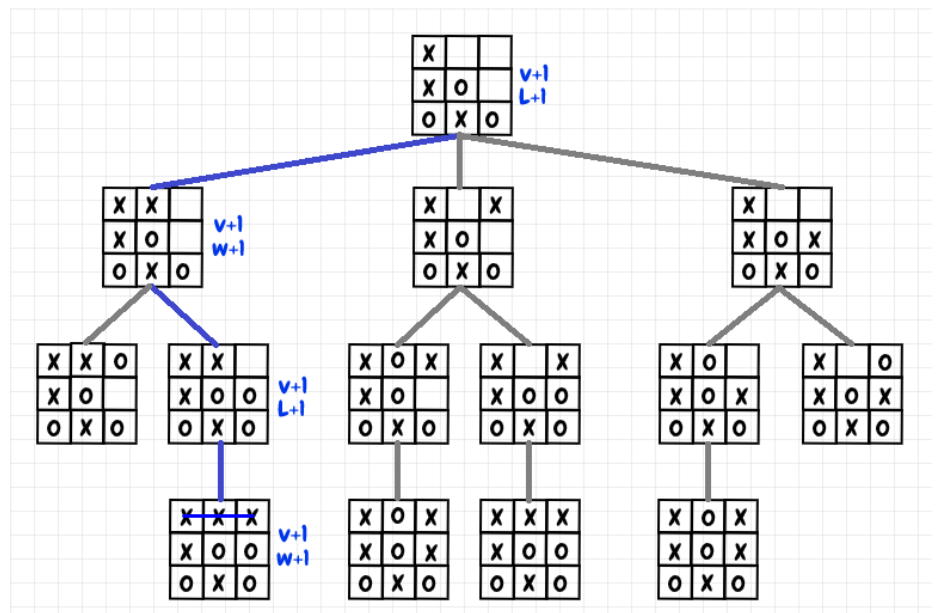
1. Minimax Concept:

- Minimax evaluates all possible moves to find the optimal move for the AI.
- It assumes the opponent also plays optimally.
- Each node in the game tree represents a possible board state.



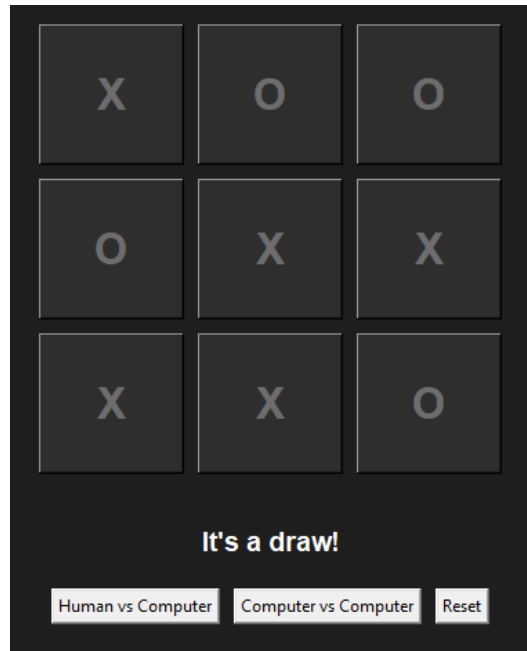
2. Alpha-Beta Pruning Concept:

- Alpha-Beta Pruning is an optimization over Minimax.
- It **skips evaluation of branches** that cannot influence the final decision.
- This reduces computation time significantly without affecting the AI's decision quality.



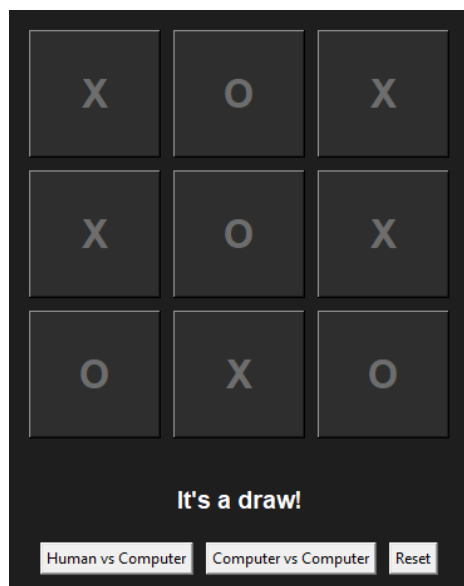
iv. Sample Input/Output (Gameplay Snapshots)

Human vs Computer Mode:



- The AI makes optimal moves using Minimax with Alpha-Beta Pruning.
- If both players play optimally, the game can end in a **draw**.

Computer vs Computer Mode:



vi. Proof of Alpha-Beta Pruning Efficiency

```
1  # Partial Board Setup
2  test_board = [
3      ['X','O',' '],
4      [' ','X',' '],
5      [' ',' ',' ']
6  ]
7
8  # Run Without Pruning
9  minimax_counter = 0
10 minimax_proof(test_board, depth=6,
11                alpha=-inf, beta=inf,
12                maximizing_player=True,
13                player_symbol=player(test_board),
14                use_pruning=False)
15 nodes_minimax = minimax_counter
16
17 # Run With Alpha-Beta Pruning
18 minimax_counter = 0
19 minimax_proof(test_board, depth=6,
20                alpha=-inf, beta=inf,
21                maximizing_player=True,
22                player_symbol=player(test_board),
23                use_pruning=True)
24 nodes_alpha_beta = minimax_counter
25
26 print("Pure Minimax:", nodes_minimax)
27 print("Alpha-Beta:", nodes_alpha_beta)
28
```

To prove that Alpha-Beta Pruning is more efficient than the pure Minimax algorithm, we conducted an experiment on a partially completed Tic Tac Toe board. The selected board state already had some moves played, which created multiple possible future game paths. We ran the Minimax algorithm twice on this state: first without pruning (pure Minimax) and then with Alpha-Beta pruning enabled. In the first run, Minimax explored all possible nodes in the game tree, whereas in the second run, Alpha-Beta Pruning skipped unnecessary branches whenever a better or worse outcome was already guaranteed. After both executions, we compared the number of nodes visited. The results showed that pure Minimax visited 324 nodes, while Alpha-Beta Pruning visited only 121 nodes, producing the same optimal move in both cases. This proves that Alpha-Beta Pruning reduces the number of computations significantly without compromising decision quality, resulting in a faster and more efficient AI for Tic Tac Toe.

vi. Comparison between Minimax and Alpha-Beta Pruning

Feature	Minimax	Alpha-Beta Pruning
Purpose	Finds the optimal move	Finds the optimal move faster
Exploration	Explores all possible moves	Skips unnecessary branches
Time Complexity	$O(b^d)$	$O(b^{(d/2)})$ on average
Memory Usage	Higher	Lower
Optimality	Always finds optimal move	Always finds optimal move
Efficiency in Practice	Slower for deep trees	Much faster, especially in large trees
Gameplay Impact	Same optimal results	Same optimal results, faster

Observation:

- Alpha-Beta pruning significantly improves computation time while producing the same results as standard Minimax.
- For Tic Tac Toe, this is not critical due to small tree size, but for larger games, pruning is essential.

vii. Conclusion and Challenges

Conclusion:

- The project successfully demonstrates **Tic Tac Toe with AI using Minimax and Alpha-Beta Pruning**.
- The AI is unbeatable in **Human vs Computer mode**.
- The **Computer vs Computer mode** highlights the efficiency of Alpha-Beta pruning.
- The game interface is **modern, interactive, and user-friendly**, with features like hover effects and winning highlights.
- This project bridges **algorithm implementation, AI decision-making, and GUI development**.

Challenges Faced:

1. Implementing **dynamic GUI updates** without freezing the interface.
2. Handling multiple **game modes** in the same application.
3. Highlighting the winning line dynamically for all possibilities (rows, columns, diagonals).
4. Integrating **Alpha-Beta pruning** into a recursive Minimax function and ensuring correctness.
5. Randomizing the first AI move without compromising optimal play in subsequent moves.